CLASE 6 · OPTIMIZACIÓN

Optimizadores: caminar mejor

Descenso por gradiente en 2D, momentum y decaimiento de la tasa de aprendizaje

github.com/datahuba/curso-redes-neuronales · Basado en el curso del Dr. J. Irving Vasquez (jivg.org)

Agenda de hoy

Tiempo	Tema
0:00	Repaso
0:10	Descenso por gradiente en 2D
0:30	Práctica: notebook 21
0:45	Valles estrechos y momentum
1:10	Práctica: notebook 22
1:25	Descanso
1:35	Decaimiento de la tasa
2:00	Práctica: notebook 23
2:20	Quiz y cierre

Al terminar la clase podrás…

1

Aplicar

el descenso con dos parámetros.

2

Leer

un mapa de contorno.

3

Explicar

el momentum y cuándo ayuda.

4

Programar

el decaimiento de la tasa.

Repaso

¿La regla?

$x \leftarrow x - \eta \cdot f'(x)$

¿η muy grande?

Oscila o diverge.

¿η muy pequeña?

Avanza muy lento.

Hoy: muchos parámetros y montañas con formas difíciles.

Descenso por gradiente en 2D

Esfera: $f(x, y) = x^2 + y^2$

$\partial f/\partial x = 2x$

$\partial f/\partial y = 2y$

Una derivada por variable (la otra queda fija).

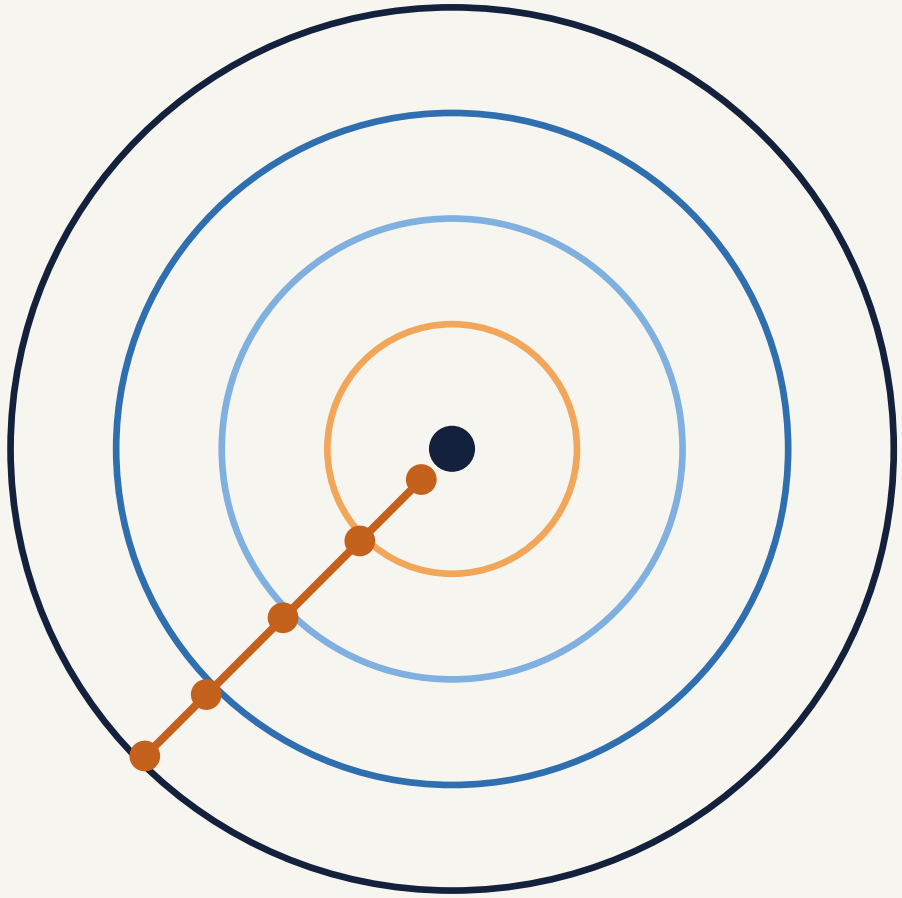
Gradiente $\nabla f = (2x, 2y)$

Una flecha que apunta hacia donde f sube más rápido.

Bajamos en contra.

$$x \leftarrow x - \eta \cdot \partial f/\partial x$$
$$y \leftarrow y - \eta \cdot \partial f/\partial y$$

El mapa de contorno



Como un mapa topográfico: cada línea une puntos de la misma “altura”.

En la esfera son círculos alrededor del mínimo (0, 0).

El camino cruza las líneas en ángulo recto: el gradiente es perpendicular a ellas.

El efecto de η en 2D (notebook 21)

η	Iteraciones	Qué pasa
0.01	300	Lento: termina en (−0.0023, −0.0023)
0.1	56	Bien
0.5	2	De un salto (solo en esta función)
0.9	66	Llega zigzagueando
1.0	300	Rebota para siempre entre (−1, −1) y (1, 1)

Cada paso multiplica x por $(1 - 2\eta)$: con η > 1 crece y diverge.

NOTEBOOKS · 15 MIN · EXPERIMENTOS

Práctica · 21_opt_gradiente_2d

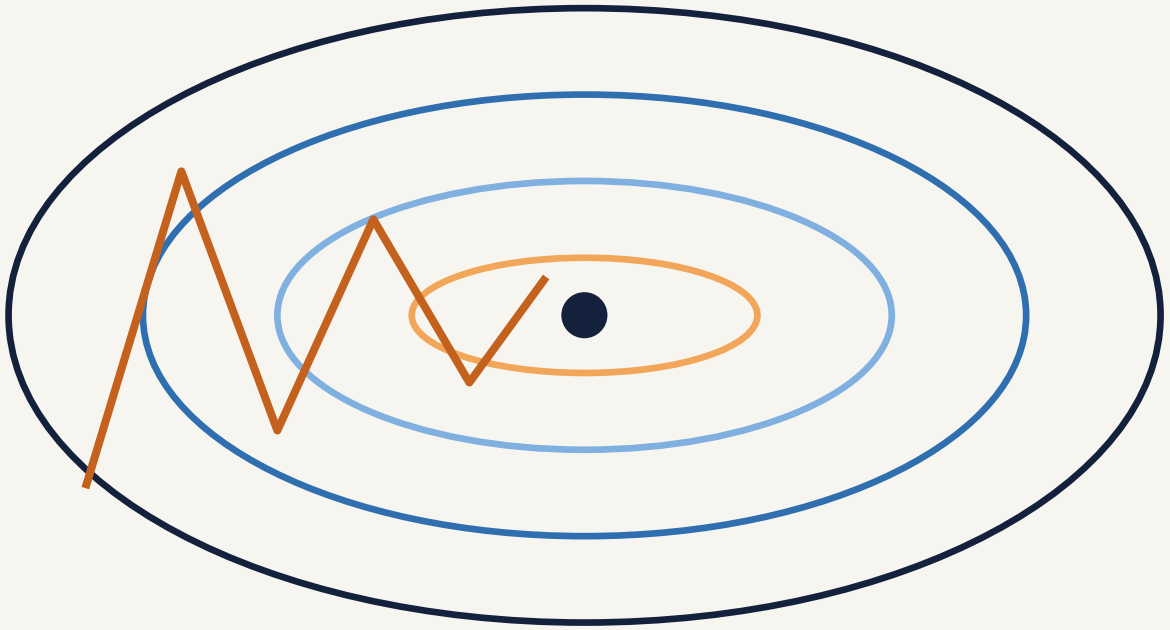
1. Ejecuta con los valores originales
2. Cambia learning_rate: 0.1, 0.5, 0.9, 1.0
3. Anota iteraciones y compáralas con la tabla

La clase GradientDescentOptimizer guarda cada paso en history: así se dibuja el camino.

```
# valores originales
Parámetros: [-0.00233, -0.00233]
Valor: 1.09e-05

# sin ffmpeg, la animación:
.save('x.gif', writer='pillow')
```

Valles estrechos: la función de Beale



Valle largo, estrecho y curvo. Mínimo en (3, 0.5).

El gradiente apunta casi siempre hacia las paredes.

Con η grande, el descenso rebota de pared a pared y puede salir disparado.

La idea del momentum

$$m \leftarrow \beta \cdot m + (1 - \beta) \cdot \nabla f$$
$$\text{parámetros} \leftarrow \text{parámetros} - \eta \cdot m$$

Promedio de gradientes

En vez del gradiente de este instante, un promedio de los recientes.

β = memoria

Con β = 0.9 el paso de ahora se parece mucho a los anteriores.

Una bola rodando por el valle

Los empujones hacia las paredes cambian de signo en cada rebote:

en el promedio se cancelan.

El empuje a lo largo del valle va siempre en la misma dirección:

se mantiene.

β	Efecto
0	Sin memoria: descenso simple
0.5 – 0.9	Suaviza los rebotes
0.99	Demasiada memoria: le cuesta girar

Resultados en Beale (notebook 22)

Desde (2.5, 1.0), 500 iteraciones

η	Descenso simple	Momentum β = 0.5	Momentum β = 0.9
0.01	$f = 6.95 \cdot 10^{-4}$	$f = 7.37 \cdot 10^{-4}$	$f = 1.07 \cdot 10^{-3}$
0.03	**diverge**	$f = 1.9 \cdot 10^{-6}$	**$f = 8.0 \cdot 10^{-7}$**

El momentum permite usar una η más grande sin descontrolarse… y llega mucho más cerca del mínimo.

NOTEBOOKS · 15 MIN · EXPERIMENTOS

Práctica · 22_opt_momentum

1. Ejecuta con los valores originales
2. Sube learning_rate a 0.03
3. Compara con el descenso simple (línea comentada)

Crea un optimizador nuevo en cada prueba: el objeto recuerda su momentum.

```
# originales (η 0.01, β 0.9)
final ≈ (2.922, 0.480)  f ≈ 1.07e-03

# η 0.03
descenso simple: OverflowError
momentum: ≈ (2.998, 0.499)
```

Descanso · 10 min

Después: el decaimiento de la tasa

El dilema de la tasa fija

η grande

Avanza rápido… pero cerca del mínimo rebota.

η pequeña

Afina bien… pero tarda muchísimo.

Decaimiento

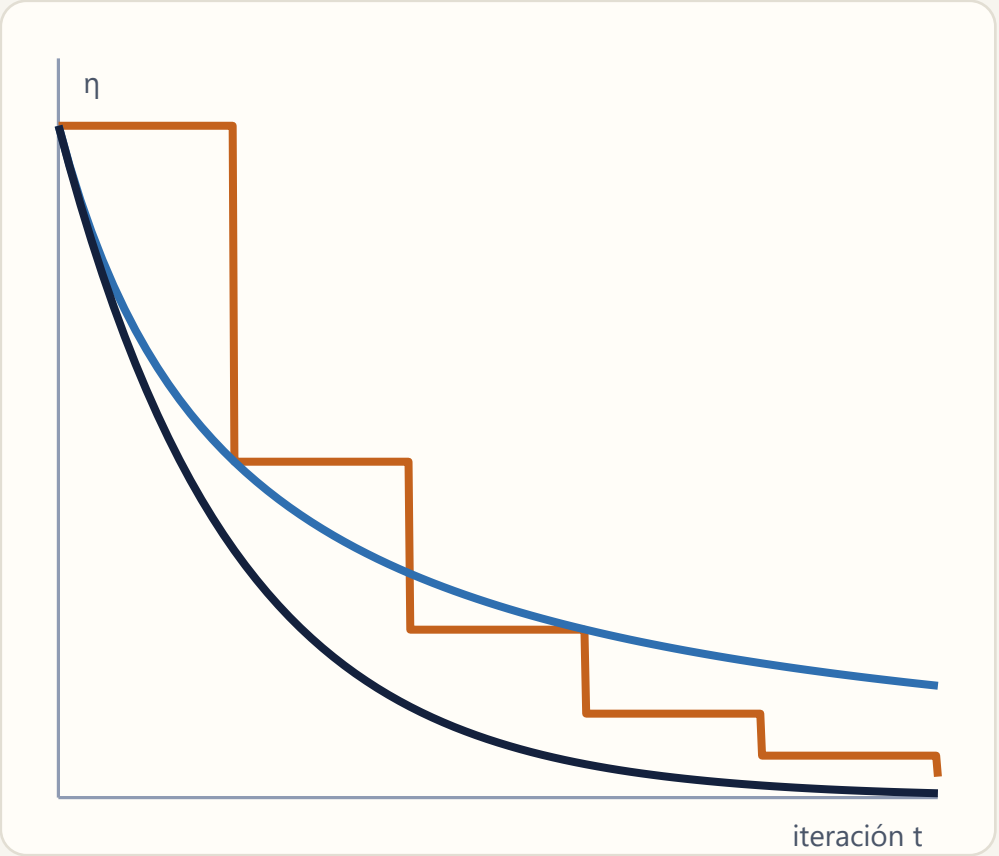
Empieza grande y la reduce con el tiempo.

Lo mejor de los dos mundos: rápido al principio, fino al final.

Tres reglas de decaimiento

Regla	Fórmula
Escalonada	$\eta_0 \cdot \gamma^{(t // s)}$
Inversa	$\eta_0 / (1 + k \cdot t)$
Exponencial	$\eta_0 \cdot e^{(-k \cdot t)}$

— escalonada — inversa — exponencial



Calcular a mano

$\eta_0 = 0.05 \cdot \gamma = 0.5$ cada $s = 100 \cdot k = 0.01$

t	Escalonada	Inversa	Exponencial
0	0.05	0.05	0.05
100	0.025	0.025	0.0184
250	0.0125	0.0143	0.0041

Escalonada en t = 250: 250 // 100 = 2 → $0.05 \cdot 0.5^2 = 0.0125$

Resultados en Beale (notebook 23)

Desde (1.0, 1.0), $\eta_0 = 0.05$, 500 iteraciones

Configuración	f final	Qué pasa
Tasa fija 0.05	$5.8 \cdot 10^{-2}$	Rebota, no llega
Escalonada	$6.3 \cdot 10^{-5}$	Casi mil veces mejor
Inversa	$1.0 \cdot 10^{-4}$	Muy bien
Exponencial	$1.5 \cdot 10^{-3}$	Decae demasiado rápido: se congela
Escalonada con $\gamma = 0.8$	**$7.8 \cdot 10^{-7}$**	La mejor

NOTEBOOKS · 20 MIN · 3 PUNTOS · NUEVO (EL ORIGINAL ESTABA VACÍO)

Práctica · 23_opt_lr_decay

TODO 1 pt las tres reglas de decaimiento

TODO 1 pt next_params con la tasa de cada iteración

TODO 1 pt experimentar y comparar

Pista: guarda la tasa en self.tasas, súmale 1 a self.t y devuelve param − tasa · gradiente.

```
# comprobaciones
t=100: 0.02500  0.02500  0.01839
t=250: 0.01250  0.01429  0.00410

fija 0.05:     f = 5.76e-02
decaimiento:   f = 6.34e-05
```

Quiz · 5 minutos

1. ¿Qué es el gradiente de f(x, y)?
2. ¿Por qué el descenso rebota en un valle estrecho?
3. ¿Qué controla β en el momentum?
4. ¿Qué ventaja dio el momentum en Beale?
5. ¿Qué pasa si la tasa decae demasiado rápido?

Para llevar

1. Más parámetros: mismo descenso, una derivada por parámetro.
2. Momentum: suaviza rebotes y permite tasas más grandes.
3. Decaimiento: empezar rápido y terminar fino.

Próxima clase: redes convolucionales, el puente al aprendizaje profundo.

Tarea: notebook 23 (3 pts) + conclusiones de 21 y 22

Leer: Goodfellow et al., Deep Learning, secciones 8.3 (momentum) y 8.5 (Adam)